Serverless Cold-Start Observatory: Project Proposal

Overview

The Serverless Cold-Start Observatory measures, explains and costs AWS Lambda cold starts, then recommends the cheapest fix that meets a latency target for each function. It combines live telemetry from CloudWatch, an XGBoost model, and a Grafana dashboard.

Problem statement

Serverless functions that sit idle start "cold": AWS must create a new environment, load the runtime and import libraries before handling the request. Users feel this as a delay of hundreds of milliseconds to several seconds.

Teams struggle with cold starts in three ways:

- **No visibility.** Cold starts are hidden in log lines, so teams only know the app is "sometimes slow".
- **Unclear causes.** Memory size, libraries, packaging and CPU architecture all affect init time, but teams usually guess which one matters.
- **Blind fixes.** SnapStart, provisioned concurrency and pre-warming each cost differently. Teams often overpay "to be safe" or ignore the problem.

Since August 2025, AWS also bills the INIT phase for standard ZIP functions (AWS Compute Blog). Cold starts now cost money as well as time.

Proposed solution

The observatory deploys a fleet of test Lambda functions, triggers cold starts in a controlled way, and turns the results into four features:

1. **Cold-start cost meter.** Converts every INIT duration into cost per function and a projected monthly figure.
2. **Import profiler.** Uses Python's built-in import timing to show which libraries cause most of the delay.
3. **arm64 vs x86 comparison.** Measures init time and cost on both CPU architectures, across memory sizes and packaging types (ZIP, layer, container image).

4. **Strategy recommender.** XGBoost predicts init time and cold-start probability. The recommender then compares on-demand, SnapStart and provisioned concurrency, and picks the cheapest option that meets a latency target.

Tech stack

Layer	Tool	Role
Compute	AWS Lambda (Python 3.12)	Test functions, collector, experiment runner
Infrastructure as code	AWS SAM	Deploys the function fleet
Scheduling	Amazon EventBridge	Runs idle-gap experiments on a schedule
Logging and metrics	Amazon CloudWatch	Captures REPORT and import-time log lines; stores custom metrics
Storage	Amazon DynamoDB	Invocation telemetry, import profiles, recommendations
Machine learning	Python, XGBoost, SHAP, scikit-learn	Init-time and cold-start probability models; feature explanations
Dashboard	Grafana (Docker or Grafana Cloud free tier)	Visualises cost, latency and recommendations

Pipeline

The project runs in five stages, each feeding the next:

1. **Build the function fleet.** Four dummy workloads (no imports, web libraries, pandas, scikit-learn) deployed with SAM across memory sizes, both architectures and three packaging types.
2. **Run experiments.** Force cold starts by updating an environment variable before each call, about 30 samples per function. EventBridge also runs idle-gap tests (1 to 40 minutes) to learn when environments are recycled.
3. **Collect and store.** A CloudWatch subscription filter sends REPORT and import-time log lines to a collector Lambda. It parses init duration, memory and module timings, then writes them to DynamoDB and publishes custom metrics.

4. **Model and recommend.** XGBoost predicts init time (as a range) and cold-start probability, validated with grouped cross-validation. SHAP explains which factors matter most. The recommender combines the predictions with AWS pricing to choose a strategy per function.
5. **Visualise.** Grafana shows cold-start rate, init time by architecture and packaging, cost per function, slowest imports, and recommended strategies.

Expected outcomes and benefits

The project delivers a working dashboard, a trained model, and a dataset comparing architectures, packaging types and memory sizes.

- **Developers** see why a function is slow, down to individual libraries.
- **Companies** spend on cold-start fixes only where they pay off, and can forecast INIT charges.
- **End users** get faster, more consistent response times.

The benefit is largest for user-facing APIs with low or spiky traffic, functions with heavy dependencies, and latency-sensitive apps such as chat and payments. For high-traffic functions, cold starts are rare and the gain is small.